Tarea: Introducción a Kubernetes

1. Resumen: ¿Qué es Kubernetes?

Qué es

Kubernetes (también llamado K8s) es una plataforma de código abierto, portable y extensible para administrar aplicaciones que corren en contenedores. Permite configurar la infraestructura de forma declarativa (se describe el estado que se quiere y el sistema se encarga de alcanzarlo) y automatizar muchas tareas operativas. El nombre viene del griego y significa "timonel" o "piloto"; la abreviatura K8s sale de las ocho letras entre la "K" y la "s". Google lo liberó como proyecto open source en 2014, basado en más de 15 años de experiencia corriendo cargas de producción con su sistema interno Borg. Hoy lo mantiene la comunidad bajo la Cloud Native Computing Foundation (CNCF).

Contexto: cómo se llegó a los contenedores

- **Despliegue tradicional:** las aplicaciones corrían directamente en servidores físicos. No había forma de limitar los recursos de cada una, así que una aplicación podía acaparar CPU o memoria y afectar a las demás. Separarlas en servidores distintos era caro y desperdiciaba hardware.
- **Virtualización:** las máquinas virtuales permitieron correr varios sistemas aislados en un mismo servidor físico, con mejor aprovechamiento y seguridad. Pero cada VM carga su propio sistema operativo completo, lo que la hace pesada.
- **Contenedores:** comparten el sistema operativo del host, por lo que son mucho más ligeros, arrancan rápido y son portables entre nubes, sistemas operativos y entornos (desarrollo, pruebas, producción).

El problema es que cuando una empresa tiene cientos o miles de contenedores, administrarlos a mano se vuelve imposible. Ahí entra Kubernetes.

Qué problemas resuelve

- **Descubrimiento de servicios y balanceo de carga:** expone los contenedores con un nombre DNS o IP y reparte el tráfico entre ellos para que el servicio se mantenga estable.
- **Orquestación de almacenamiento:** monta automáticamente el almacenamiento que se necesite (local, en la nube, etc.).
- **Despliegues y rollbacks automáticos:** se define el estado deseado y Kubernetes cambia el estado actual a un ritmo controlado; si algo sale mal, se puede regresar a la versión anterior.
- **Bin packing automático:** acomoda los contenedores en los nodos del clúster según la CPU y memoria que cada uno pide, aprovechando mejor los recursos.
- **Autorreparación:** reinicia contenedores que fallan, reemplaza los que no responden a los health checks y no les manda tráfico hasta que estén listos.
- **Manejo de secretos y configuración:** guarda contraseñas, tokens y llaves de forma segura y permite actualizarlos sin reconstruir las imágenes.
- **Escalado horizontal:** aumenta o reduce el número de réplicas con un comando o de forma automática según el uso de CPU.

- **Trabajos batch y CI:** además de servicios, también administra tareas por lotes.
- **Extensibilidad:** se le pueden agregar funcionalidades sin modificar su código fuente.

Qué no es Kubernetes

No es un PaaS completo "todo incluido": no compila código fuente, no trae bases de datos, colas de mensajes ni cachés integrados, y no impone una herramienta específica de logs, monitoreo o configuración. Tampoco es solo un orquestador que sigue pasos fijos (A, luego B, luego C); funciona como un conjunto de procesos de control independientes que llevan continuamente el estado actual hacia el estado deseado, lo que lo hace más robusto y fácil de extender.

En qué casos se utiliza

- Aplicaciones con **arquitectura de microservicios**, donde hay muchos servicios pequeños que deben desplegarse y escalarse por separado.
- Sistemas que necesitan **alta disponibilidad** y tolerancia a fallos (si un nodo cae, la carga se mueve a otro).
- Aplicaciones con **tráfico variable** que necesitan escalar automáticamente.
- Empresas que quieren **portabilidad entre nubes** o nube híbrida (AWS, Google Cloud, Azure u on-premise) sin reescribir su infraestructura.
- Equipos que hacen **despliegues frecuentes** con CI/CD y necesitan rollouts y rollbacks seguros.
- Cargas de **procesamiento de datos, machine learning y trabajos batch**.

2. Empresas que usan Kubernetes

Spotify

Spotify fue de las primeras empresas en adoptar microservicios y Docker. Durante años corrió sus servicios en contenedores sobre una flota de máquinas virtuales usando **Helios**, un orquestador de contenedores que ellos mismos desarrollaron. A finales de 2017 el equipo de infraestructura concluyó que mantener Helios con un equipo pequeño no era eficiente comparado con usar una herramienta respaldada por una comunidad mucho más grande, así que decidieron migrar a Kubernetes. El objetivo era ganar velocidad de desarrollo, reducir costos y alinearse con las prácticas estándar de la industria.

La migración fue gradual: durante un tiempo los servicios corrieron en Helios y en Kubernetes al mismo tiempo, para no "poner todos los huevos en una sola canasta" hasta comprobar que Kubernetes aguantaba la carga y el estrés de producción. Con el tiempo movieron más de 150 microservicios a Kubernetes.

Beneficios:

- La planeación de capacidad pasó de tardar casi una hora a resolverse en segundos o minutos, porque Kubernetes asigna recursos automáticamente.
- Los equipos dedican menos tiempo a aprovisionar capacidad manualmente y más tiempo a construir funcionalidades para el producto.

- Mejor uso de CPU gracias al bin packing y a la posibilidad de compartir nodos entre varios servicios (multi-tenancy).
- Al usar una herramienta estándar, aprovechan el ecosistema y la comunidad en vez de mantener su propio orquestador.

Airbnb

Airbnb empezó como una aplicación monolítica en Ruby on Rails. Conforme creció la empresa y el número de ingenieros, el monolito se volvió difícil de mantener y desplegar, por lo que decidieron migrar a una arquitectura orientada a servicios (SOA) con cientos de microservicios. El reto era que cada servicio necesitaba su propio ciclo de entrega continua, y la infraestructura que tenían (basada en configuraciones manuales con Chef sobre instancias de AWS) no escalaba bien. Como explicó Melanie Cebula, ingeniera de infraestructura de Airbnb, necesitaban "escalar la entrega continua horizontalmente".

Para resolverlo adoptaron Kubernetes con varios clústeres y construyeron herramientas internas encima (como **kube-gen**, que genera los archivos de configuración de Kubernetes a partir de plantillas) para que cualquier ingeniero pudiera crear y desplegar un servicio sin ser experto en infraestructura. La configuración de cada servicio vive en el mismo repositorio que su código y pasa por el mismo proceso de revisión.

Beneficios:

- Los despliegues crecieron de forma exponencial; según la propia Airbnb llegaron a unos 125,000 despliegues a producción por año.
- Los equipos de producto son autónomos: pueden crear servicios nuevos y desplegarlos sin depender del equipo de infraestructura.
- Cada servicio puede tener varios entornos (canary, staging, pruebas de carga) de forma estandarizada.
- Mejor aprovechamiento de los recursos de la nube y escalado automático según la demanda de los usuarios.

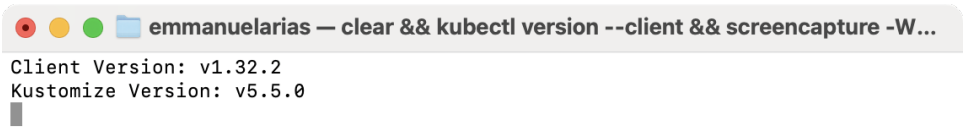
3. Instalación de kubectl

Comando ejecutado:

```
kubectl version --client
```

Salida:

```
Client Version: v1.32.2
Kustomize Version: v5.5.0
```



Fuentes

- Kubernetes Documentation, "Overview": https://kubernetes.io/docs/concepts/overview/
- Kubernetes Case Study, Spotify: https://kubernetes.io/case-studies/spotify/
- CNCF Blog, "With Kubernetes, Spotify's capacity planning went from almost an hour to seconds or minutes" (cncf.io/blog, 2019)
- Melanie Cebula, "Developing Kubernetes Services at Airbnb Scale" (InfoQ): https://infoq.com/presentations/airbnb-kubernetes-services
- Altoros, "Airbnb Deploys 125,000+ Times per Year with Multicluster Kubernetes" (altoros.com/blog)